Using Scrum Framework for Spec-Version Commits

A practical rule set for Kiste: backlog item -> spec change -> implementation -> test -> audit trail

Core Scrum Rule

Every meaningful Product Backlog Item should trace to:

```
Backlog Item -> Spec Change -> Implementation -> Test -> Audit Note
```

Rule: A Sprint item is not Done unless the related spec version, implementation, test, and audit trail are committed together or clearly linked.

Scrum Workflow

Scrum Event	Kiste Action	Output
Product Backlog	Declare the expected spec impact for every story.	Story includes acceptance criteria and expected version impact.
Sprint Planning	Classify the item as no spec change, patch, minor, or major.	Team understands migration, test, and audit cost before committing.
During Sprint	Commit spec, implementation, test, and audit updates with the same story ID.	Traceable change chain from requirement to delivery.
Sprint Review	Show both the feature and the spec evolution.	Stakeholders see what changed in the system contract.
Retrospective	Check whether any implementation escaped the spec process.	Process improvement for trust, auditability, and delivery speed.

Product Backlog Item Template

```
Story: Add rollback support for failed deployment

Spec impact: minor
Expected version: deploy-spec 0.3.x -> 0.4.0
Acceptance criteria:
- Rollback condition is defined
- Retry limit is defined
- Runtime implementation follows the spec
- Test proves failed rollout triggers rollback
- Audit log records rollback reason
```

Version Impact Rule

Impact	Version Bump	Meaning	Examples
Patch	0.3.1 -> 0.3.2	Clarifies existing behavior without changing it.	Typos, wording cleanup, examples, ambiguity fixes.
Minor	0.3.2 -> 0.4.0	Adds compatible behavior.	Optional fields, new backend, new workflow step, new integration.
Major	0.4.0 -> 1.0.0	May break old implementations.	Removed fields, changed defaults, changed security model, deployment contract change.

Commit Pattern During the Sprint

Use one story ID across spec, implementation, test, and audit commits.

```
spec(deploy): define rollback behavior

Version: deploy-spec 0.3.2 -> 0.4.0
Impact: minor
Story: KIS-124
Reason: Failed rollouts need deterministic recovery behavior.

impl(deploy): add rollback executor

Spec: deploy-spec 0.4.0
Story: KIS-124

test(deploy): verify rollback after failed rollout

Spec: deploy-spec 0.4.0
Story: KIS-124

audit(deploy): record rollback decision fields

Spec: deploy-spec 0.4.0
Story: KIS-124
```

Definition of Done for Kiste

- Spec impact is declared.
- Spec version is updated when behavior changes.
- Implementation references the spec version.
- Tests prove the implementation follows the spec.

- Migration exists for breaking changes.
- Audit trail is updated.
- Product Owner accepts the behavior.

Sprint Review Output

```
Sprint 5 delivered:

deploy-spec: 0.3.2 -> 0.4.0
workspace-spec: 0.2.0 -> 0.2.1
permission-spec: unchanged

New behavior:
- Failed rollout now has standard rollback
- Workspace wording clarified
- No permission behavior changed
```

Sprint Retrospective Questions

- Did any implementation happen without a spec?
- Did any spec change create hidden migration work?
- Were tests tied clearly to spec versions?
- Was the audit trail useful?
- Could the Product Owner understand the spec change?

Best Rule for Kiste

Every Sprint must produce a potentially shippable, spec-versioned, auditable increment.

No implementation change without a spec link. No breaking spec change without migration. No deployment without audit trail.